

Appendix Overview

- Appendix A** Training Procedure
- Appendix B** Dataset Details and Preprocessing
- Appendix C** Baseline and Ablation Details
- Appendix D** Implementation and Hyperparameters
- Appendix E** Linear Evaluation and Efficiency Protocol
- Appendix F** Complexity and Resource Analysis

A Training Procedure

Symmetric prefix contrastive loss. The implementation uses the symmetric form of cross-view InfoNCE. For a prefix of dimension d_i , let $\mathbf{S}^{(i)} = \text{norm}(\mathbf{H}_1^{(i)}) \text{norm}(\mathbf{H}_2^{(i)})^\top$ and let $\mathbf{y} = (1, \dots, B)$ denote the diagonal positive-pair indices. The implemented loss is

$$\ell_{\text{NCE}}^{(i)} = \frac{1}{2} \left[\text{CE}(\mathbf{S}^{(i)}/\tau, \mathbf{y}) + \text{CE}((\mathbf{S}^{(i)})^\top/\tau, \mathbf{y}) \right]. \quad (14)$$

Thus, both augmented views act as anchors during standard MRL training.

Warm-up stage. The warm-up objective implemented in the released code is the average of the seven prefix losses:

$$\mathcal{L}_{\text{warm}} = \frac{1}{K} \sum_{i=1}^K \ell_{\text{NCE}}^{(i)}. \quad (15)$$

This stage gives every prefix a discriminative initialization before cross-dimensional signals are introduced.

Main stage. After warm-up, the code constructs the CDMD relational evidence by flattening the cross-view cosine-similarity matrix and applying ReLU. All lower-dimensional prefixes are compared with the $d_K = 768$ prefix. The implementation treats the resulting vectors as non-negative relational evidence and evaluates the two stop-gradient relative-entropy directions directly; it does not apply a softmax normalization to these vectors. For HPEM, the smallest prefix uses uniform symmetric InfoNCE. For every $i > 1$, the confusion scores computed at d_{i-1} are detached, converted into weights that sum to one, and used to aggregate the per-anchor InfoNCE losses at d_i . DALs supplies both the adaptive temperature $\tau_i = \tau_0 \exp(\phi_1 d_i/d_K + \phi_2)$ and the dimension weight $\alpha_i = \exp(\lambda d_i/d_K)$.

B Dataset Details

OGBN-ARXIV. OGBN-ARXIV is a paper citation network in which nodes represent computer-science papers and directed edges represent citations. Each node is associated with a 128-dimensional feature vector derived from the title and abstract of the corresponding paper, and the prediction target is one of 40 subject areas. Its relatively high edge and node homophily make it a representative homophilous citation graph.

OGBN-MAG. OGBN-MAG originates from the heterogeneous Microsoft Academic Graph. Following the node-classification setting used in our experiments, we retain the 736,389 paper nodes and the 10,792,672 paper-to-paper citation edges. Paper features are 128-dimensional, and the

Algorithm 1: Training procedure of MCNE

Require: Graph $\mathcal{G}$, dimensions $\mathcal{D} = \{32, 64, 128, 256, 384, 512, 768\}$, warm-up epochs T_w , main-stage epochs T_m

- 1: Convert the graph to an undirected graph and ℓ_2 -normalize each node-feature vector
- 2: **for** $t = 1$ to $T_w + T_m$ **do**
- 3: **for** each neighbor-sampled mini-batch $\mathcal{B}$ **do**
- 4: Independently drop edges to obtain $\mathcal{G}_1^{\mathcal{B}}$ and $\mathcal{G}_2^{\mathcal{B}}$
- 5: Encode both views into $\mathbf{H}_1, \mathbf{H}_2 \in \mathbb{R}^{B \times 768}$
- 6: **if** $t \leq T_w$ **then**
- 7: Compute $\mathcal{L}_{\text{warm}}$ using Eq. 15
- 8: **else**
- 9: Compute CDMD between every $d_i < 768$ and $d_K = 768$
- 10: Compute uniform InfoNCE at $d_1 = 32$
- 11: **for** $i = 2$ to K **do**
- 12: Compute detached confusion margins at d_{i-1}
- 13: Compute HPEM-weighted InfoNCE at d_i
- 14: **end for**
- 15: Apply $\{\tau_i, \alpha_i\}_{i=1}^K$ from DALs
- 16: Combine the weighted HPEM objectives and CDMD
- 17: **end if**
- 18: Update the encoder, β, ϕ_1, ϕ_2 , and λ with Adam
- 19: **end for**
- 20: **end for**
- 21: **return** One 768-dimensional encoder whose prefixes are evaluated independently

task is to predict one of 349 publication venues. Compared with the other two datasets, OGBN-MAG has substantially lower edge and node homophily and therefore provides a more challenging setting for learning discriminative node representations.

OGBN-PRODUCTS. OGBN-PRODUCTS is an Amazon product co-purchasing network. Nodes represent products, edges indicate that two products are frequently purchased together, and the 100-dimensional node features encode product information. The node-classification task assigns each product to one of 47 categories. This dataset is the largest graph in our evaluation, containing approximately 2.45 million nodes and 61.86 million edges.

For all three datasets, we use the official training, validation, and test splits provided by OGB (Hu et al. 2020). For OGBN-MAG, only paper nodes and the paper-cites-paper relation are retained before this conversion. No class labels are used when training the representation models. Labels are used only for fitting and evaluating the downstream linear classifier.

C Baseline and Ablation Details

We compare MCNE with six representative graph self-supervised learning methods. **DGI** (Veličković et al. 2019) maximizes mutual information between node representations and a graph-level summary. **GraphCL** (You et al. 2020) was

Dataset	#Nodes	#Edges	Avg. Deg.	#Feat.	#Classes	$\mathcal{H}_e$	$\mathcal{H}_n$
OGBN-ARXIV	169,343	1,166,243	6.9	128	40	0.65	0.64
OGBN-MAG	736,389	10,792,672	14.7	128	349	0.30	0.31
OGBN-PRODUCTS	2,449,029	61,859,140	25.4	100	47	0.81	0.82

Table 4: Raw benchmark statistics of the three large-scale OGB datasets before the undirected conversion used by the encoder. $\mathcal{H}_e$ and $\mathcal{H}_n$ denote edge homophily and node homophily, respectively.

originally designed for graph-level representation learning. To adapt it to the downstream node classification task considered in this work, we implement a node-level variant that treats the seed nodes in each neighbor-sampled subgraph as contrastive instances. Specifically, two augmented views are generated for each sampled subgraph, and representations of the same surviving seed node across the two views form a positive pair, while the other surviving seed nodes serve as negatives. This adaptation retains GraphCL’s GIN encoder, projection head, and graph augmentation strategies while enabling it to learn node representations for node classification. **GRACE** (Zhu et al. 2020) performs node-level contrastive learning between two stochastically augmented views. **CCA-SSG** (Zhang et al. 2021) replaces explicit negative pairs with an invariance and feature-decorrelation objective. **SSGE** (Liu et al. 2025) is a negative-free method that encourages cross-view alignment while matching the learned representation distribution to an isotropic Gaussian. **IrGAE** (Li et al. 2026) constructs asymmetric left and right graph views and learns node representations by reconstructing graph structure or node features between them, thereby performing implicit contrastive learning.

The six conventional baselines produce representations at a single fixed dimension. Consequently, each of them is trained independently for every target dimension in

$$\mathcal{D} = \{32, 64, 128, 256, 384, 512, 768\}.$$

For a resource-adaptive comparison, we additionally construct the **MRL** baseline by applying the standard multi-prefix objective in Eq. 3 to the GCL encoder. **MRL** and **MCNE** both produce a 768-dimensional representation and are trained once; the representation at dimension d_i is obtained from its first d_i coordinates. This protocol separates gains brought by the proposed dimension-aware components from gains that arise from jointly training multiple prefixes.

Ablation configurations. The ablation configurations correspond directly to those reported in Table 3. **MRL** denotes the standard multi-prefix contrastive baseline. **+ CDMD** enables CDMD only. **+ HPEM** enables HPEM only. **+ HPEM + DALs** enables HPEM together with DALs. The complete **MCNE** configuration enables CDMD, HPEM, and DALs. All other dataset-specific settings are held fixed.

D Implementation

Environment and encoder. The reproduction package requires Python 3.10 or later, PyTorch 2.0 or later, PyTorch Geometric 2.7 or later, and the OGB package. The reported experiments use a GCN encoder with ReLU activations and

batch normalization. The full output dimension is 768, the encoder dropout is zero. Training uses neighbor sampling over all nodes, including unlabeled nodes, because representation learning is fully self-supervised. All models are optimized with Adam. The experiments reported in the paper were conducted on an NVIDIA Tesla V100-SXM2 GPU with 32 GB of memory.

View generation. For **MCNE**, **MRL**, **GRACE**, **CCA-SSG**, and **SSGE**, graph views are generated online for every neighbor-sampled mini-batch. The **MCNE** configuration does not mask node features; its two views are generated by independently dropping edges with the dataset-specific probabilities in Table 5. The augmentation is applied after neighbor sampling.

Optimization schedule. The warm-up stage uses Adam with learning rate 10^{-3} . At the first main-stage epoch, the same optimizer state is retained and the learning rate is changed to 10^{-5} . Self-supervised early stopping is disabled in the reproduction commands, so every run completes the fixed number of epochs shown in Table 5. The representation model is initialized and trained with seed 0.

E Linear Evaluation and Efficiency Protocol

Linear evaluation. After self-supervised training, we freeze the encoder and crop the learned representation to each target dimension. For each resulting prefix, we train a separate linear classifier on the official training split for 200 epochs using Adam with a learning rate of 10^{-2} , zero weight decay, cross-entropy loss, and a batch size of 8192. We repeat the linear evaluation with seeds 0, 1, 2, 3, and 4 using the same frozen representations and report the mean and standard deviation of test accuracy across the five runs.

Efficiency measurement. The wall-clock results in Table 2 measure the time required to obtain representations at all seven target dimensions. For **GRACE**, this is the total time of seven independently trained models. For **MRL** and **MCNE**, it is the time of one multi-prefix training run. The comparison is made within each dataset because dataset scale and epoch counts required for convergence differ across datasets.

F Complexity and Resource Analysis

Let B be the number of seed nodes in a sampled mini-batch, K the number of prefixes, and d_K the full embedding dimension. The GCN cost depends on the sampled computation graph and is shared by all prefixes because the encoder is evaluated only once per view. For each prefix, the contrastive

Setting	OGBN-ARXIV	OGBN-MAG	OGBN-PRODUCTS
GCN layers	3	2	2
Hidden/output dimension	768	768	768
Dropout	0.0	0.0	0.0
Training mode	Neighbor sampling	Neighbor sampling	Neighbor sampling
Training fanouts	(10, 10, 10)	(10, 10)	(10, 10)
Training batch size	4096	4096	4096
Warm-up epochs T_w	200	50	20
Main-stage epochs T_m	200	50	20
Warm-up learning rate	10^{-3}	10^{-3}	10^{-3}
Main-stage learning rate	10^{-5}	10^{-5}	10^{-5}
Weight decay	10^{-5}	10^{-5}	10^{-5}
Feature masking, views 1/2	0/0	0/0	0/0
Edge dropping, views 1/2	0.4/0.4	0.6/0.6	0.5/0.5
CDMD weight μ	1.0	1.0	0.1
Inference fanouts	(-1, -1, -1)	(10, 10)	(10, 10)
Inference batch size	256	256	4096

Table 5: Dataset-specific settings for MCNE. A fanout of -1 means all neighbors are used during inference.

MCNE setting	Value
Prefix dimensions	32, 64, 128, 256, 384, 512, 768
MRL loss weight	1.0
Base InfoNCE temperature τ	0.1
HPEM base temperature τ_0	0.1
Initial HPEM scale β_0	0.1
Initial ϕ_1, ϕ_2	0, 0
Initial DALs parameter λ	0
CDMD reference	768-dimensional prefix
CDMD evidence transform	ReLU

Table 6: MCNE settings shared by all three datasets.

components construct a $B \times B$ cross-view similarity matrix. Consequently, the prefix objectives have time complexity

$$O\left(B^2 \sum_{i=1}^K d_i\right), \quad (16)$$

in addition to the two GCN forward passes. CDMD computes the full-dimensional reference similarity once and one similarity matrix for every lower-dimensional prefix. HPEM computes a confusion matrix at the preceding prefix and a contrastive matrix at the current prefix. Both therefore have the same leading pairwise-similarity order as multi-prefix InfoNCE. The pairwise matrices require $O(B^2)$ storage individually; retaining the autograd graphs for all prefix losses gives a worst-case $O(KB^2)$ auxiliary-memory term.

At deployment, one encoder pass produces the full d_K -dimensional representation, after which a prefix is obtained by slicing. For N nodes stored in 32-bit floating point, the embedding storage at dimension d_i is

$$4Nd_i \text{ bytes.} \quad (17)$$

Thus, the 32-dimensional prefix uses $24\times$ less embedding storage than the 768-dimensional representation. Downstream linear-classification and similarity-search costs also

scale linearly with d_i . The current implementation still computes the 768-dimensional GCN output before cropping, so the principal deployment savings concern embedding storage, transfer, and downstream processing rather than the GCN encoder pass itself.